



US 20250278609A1

(19) **United States**

(12) **Patent Application Publication**
MEISTER et al.

(10) **Pub. No.: US 2025/0278609 A1**

(43) **Pub. Date: Sep. 4, 2025**

(54) **LOOP OPTIMIZATION-BASED APPROACH
FOR TASK SCHEDULING IN GRAPH
MACHINE LEARNING MODELS**

Publication Classification

(51) **Int. Cl.**
G06N 3/0464 (2023.01)

(52) **U.S. Cl.**
CPC **G06N 3/0464** (2023.01)

(71) Applicant: **QUALCOMM Incorporated**, San
Diego, CA (US)

(72) Inventors: **Benoit Jean Nicolas MEISTER**, San
Diego, CA (US); **Xiaolong LI**, San
Diego, CA (US); **Muntasir Amin
MALLICK**, North York (CA);
Chandan REDDY, Ville-en-Vermois
(FR); **Pranav SHRESTHA**, San Diego,
CA (US); **Li HE**, San Diego, CA (US)

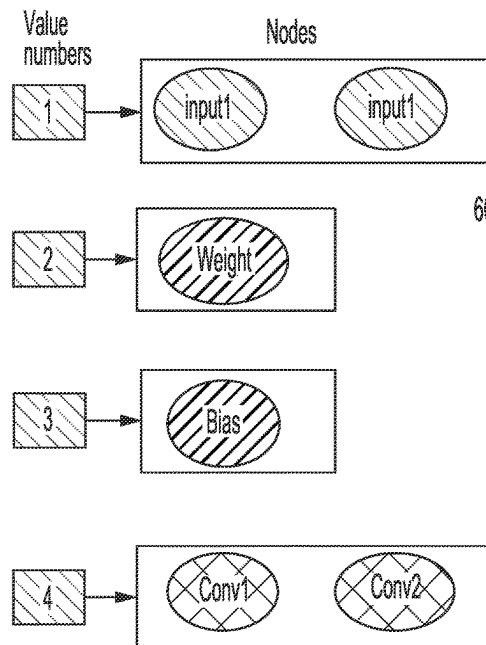
(21) Appl. No.: **18/592,222**

(22) Filed: **Feb. 29, 2024**

(57) **ABSTRACT**

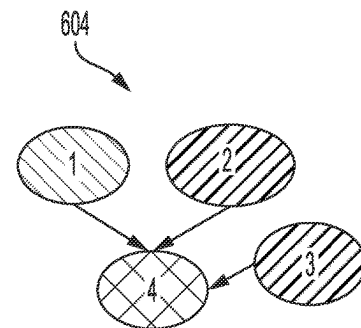
A processor-implemented method for representing compu-
tation in a machine learning (ML) model as loops includes
receiving the ML model. The ML model is initially repre-
sented as a graph having multiple nodes coupled by edges.
A value number is assigned to each node in the graph based
on a similarity in characteristics of the multiple nodes. A
loop for computations in the graph is reconstructed based on
the value number. The loop bounds are determined using an
affine scalar evolution analysis technique.

600



602

Reroll



```
for (i = start??; i < final??; i += stride??);  
  R[i] = Conv_Op(Input[i], Weight, Bias)
```

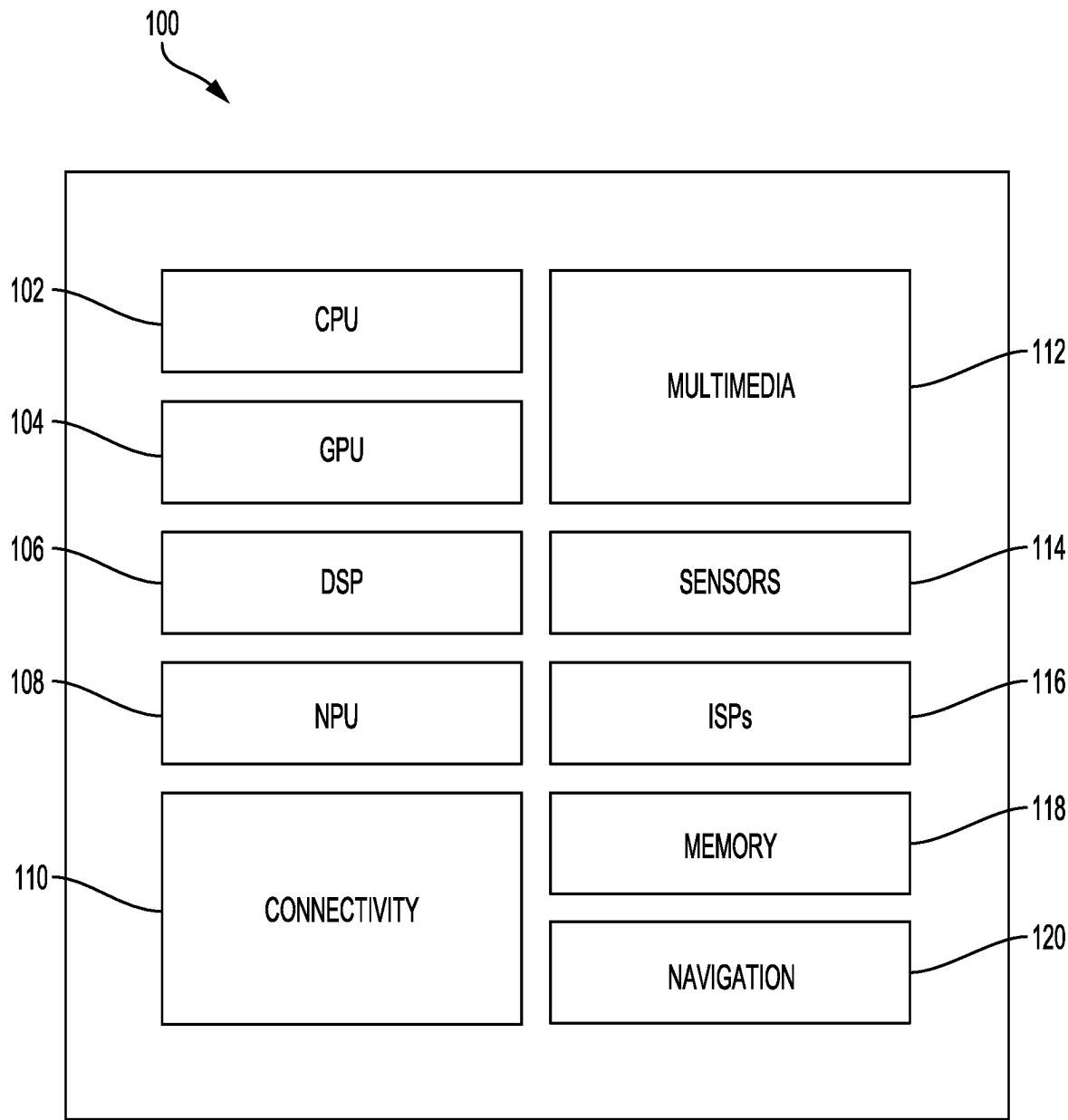


FIG. 1

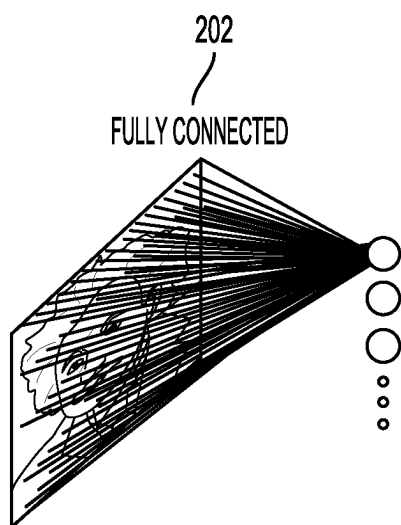


FIG. 2A

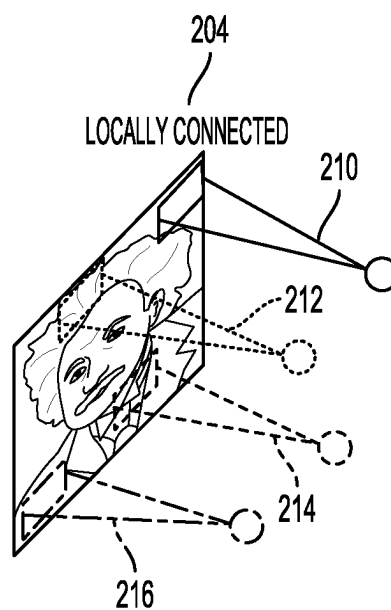


FIG. 2B

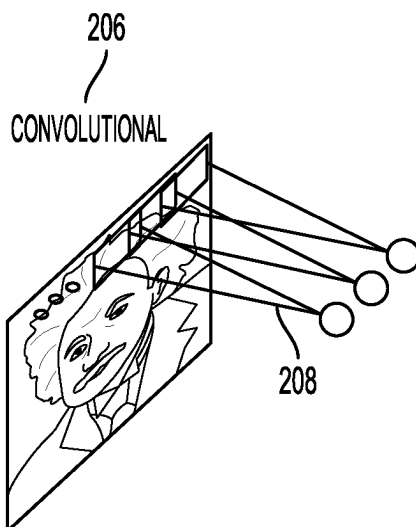


FIG. 2C

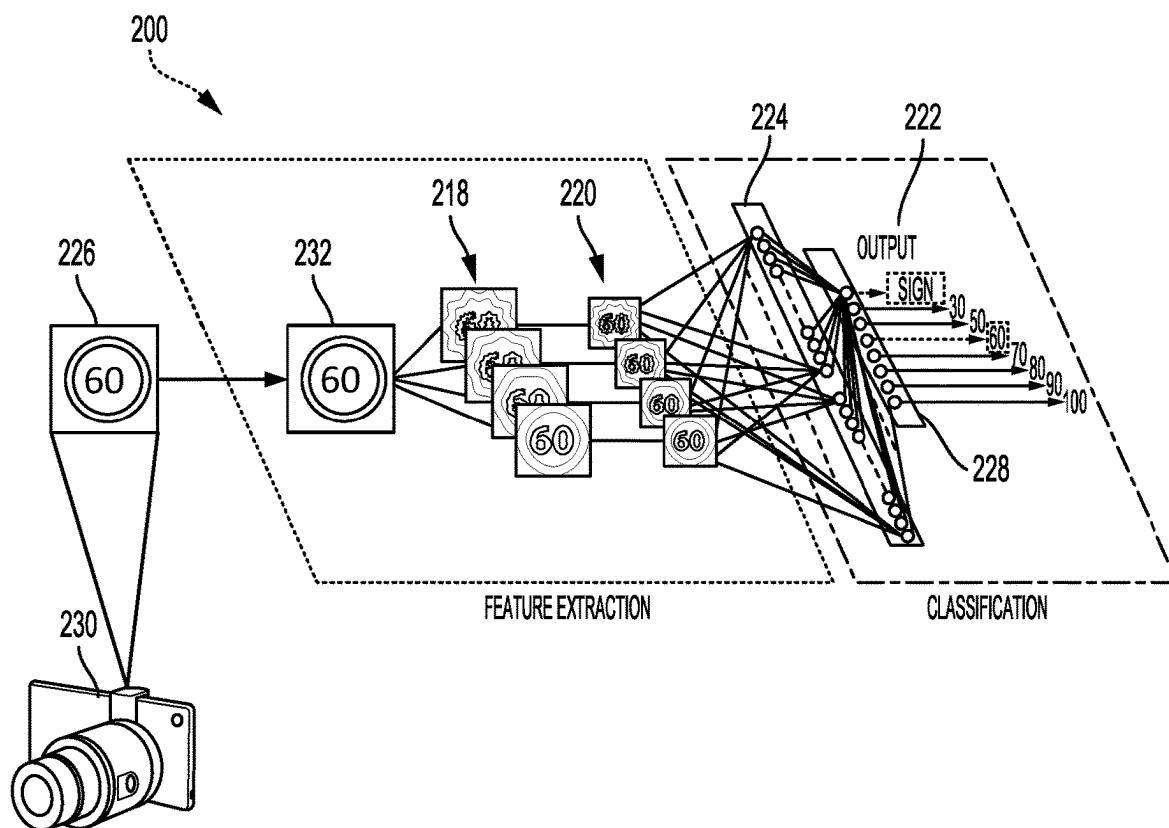


FIG. 2D

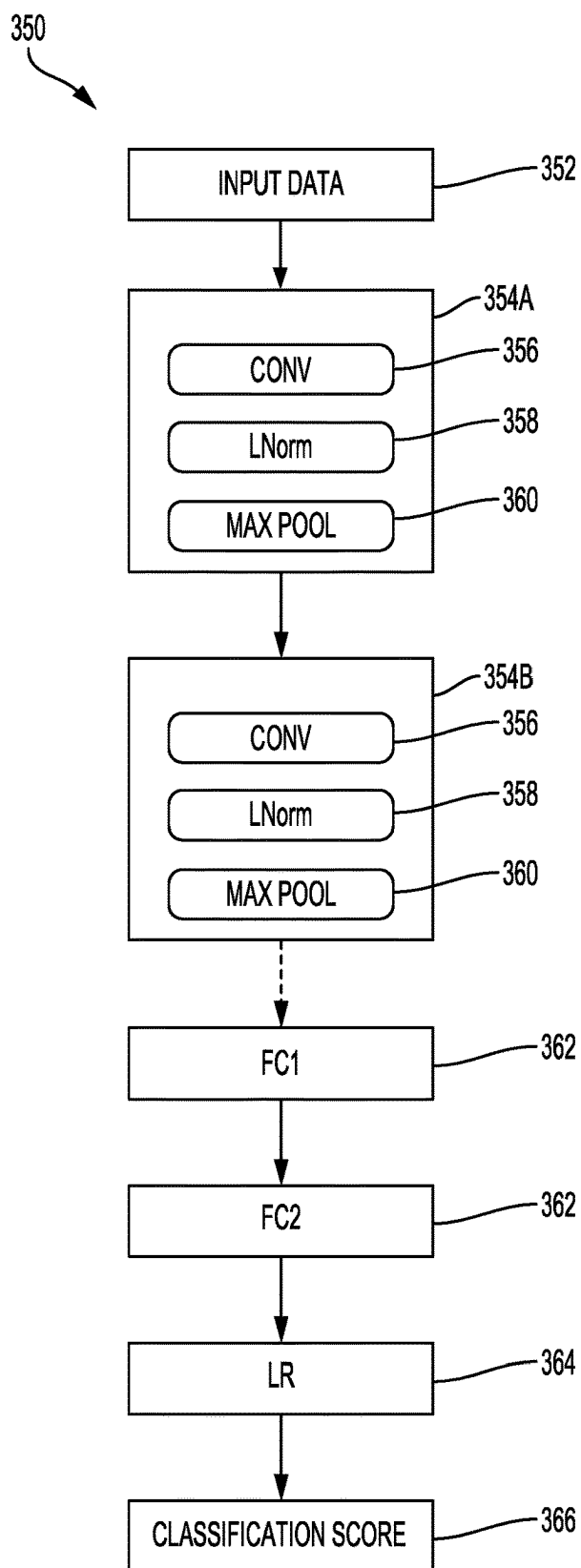


FIG. 3

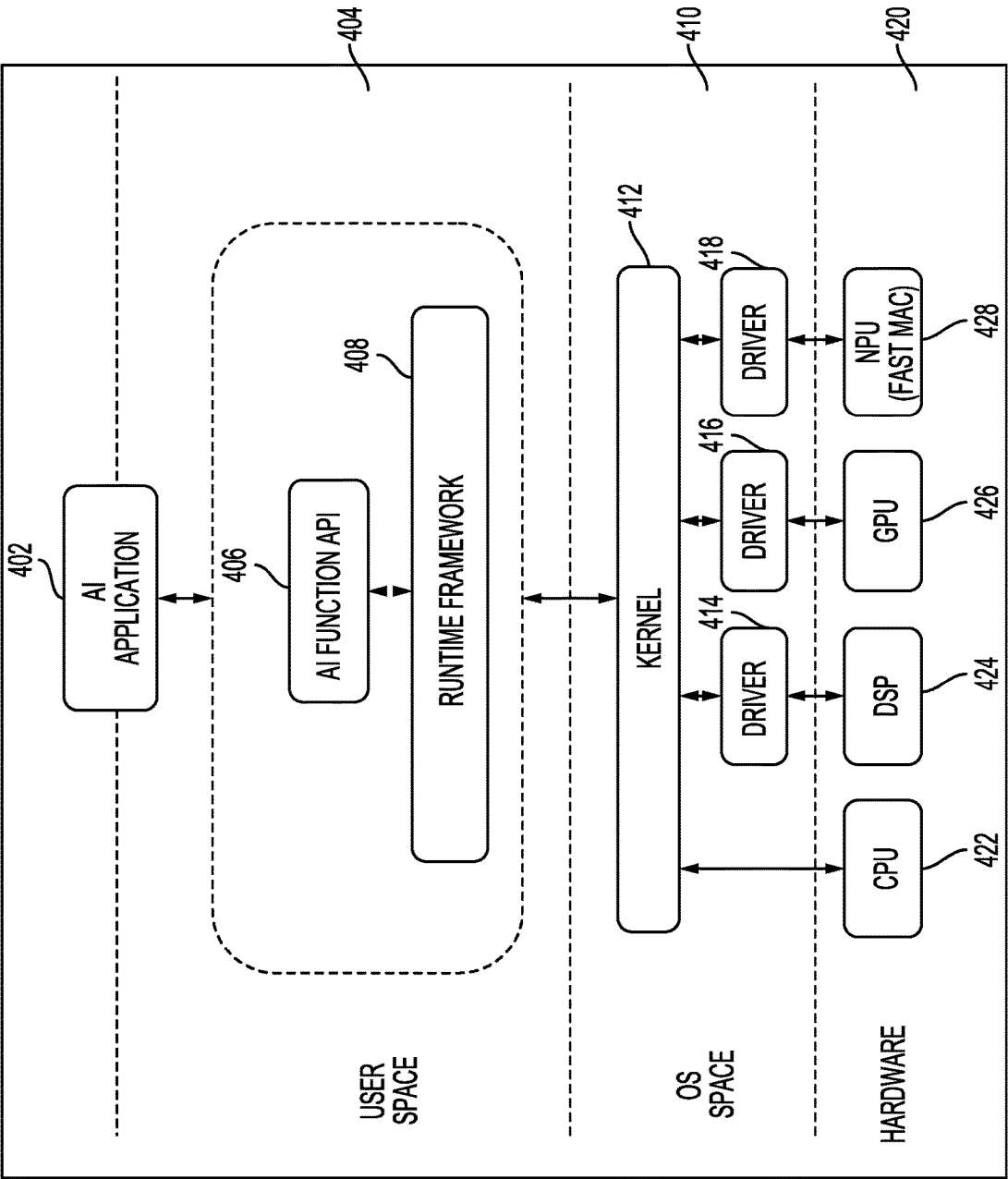


FIG. 4

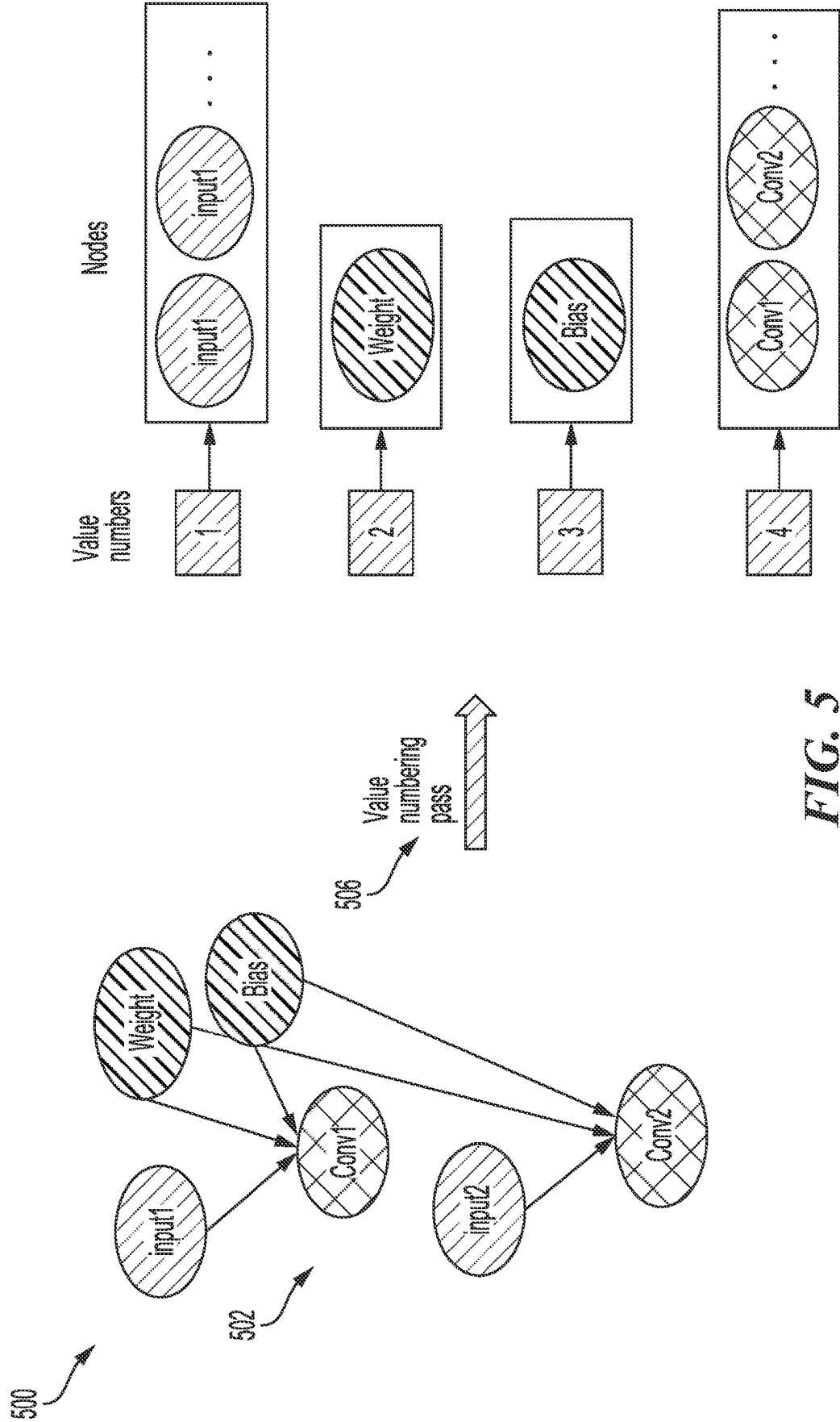


FIG. 5

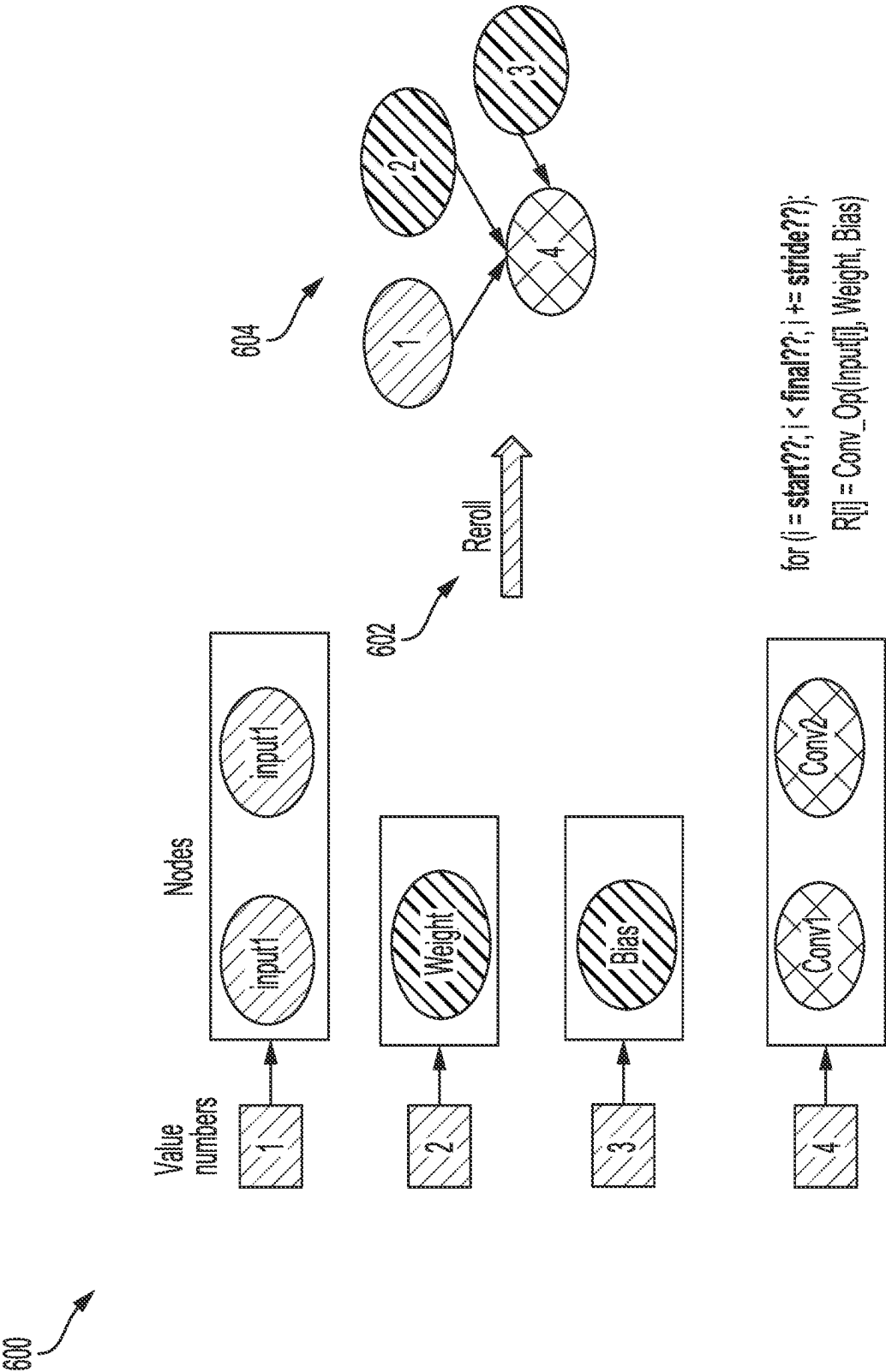


FIG. 6

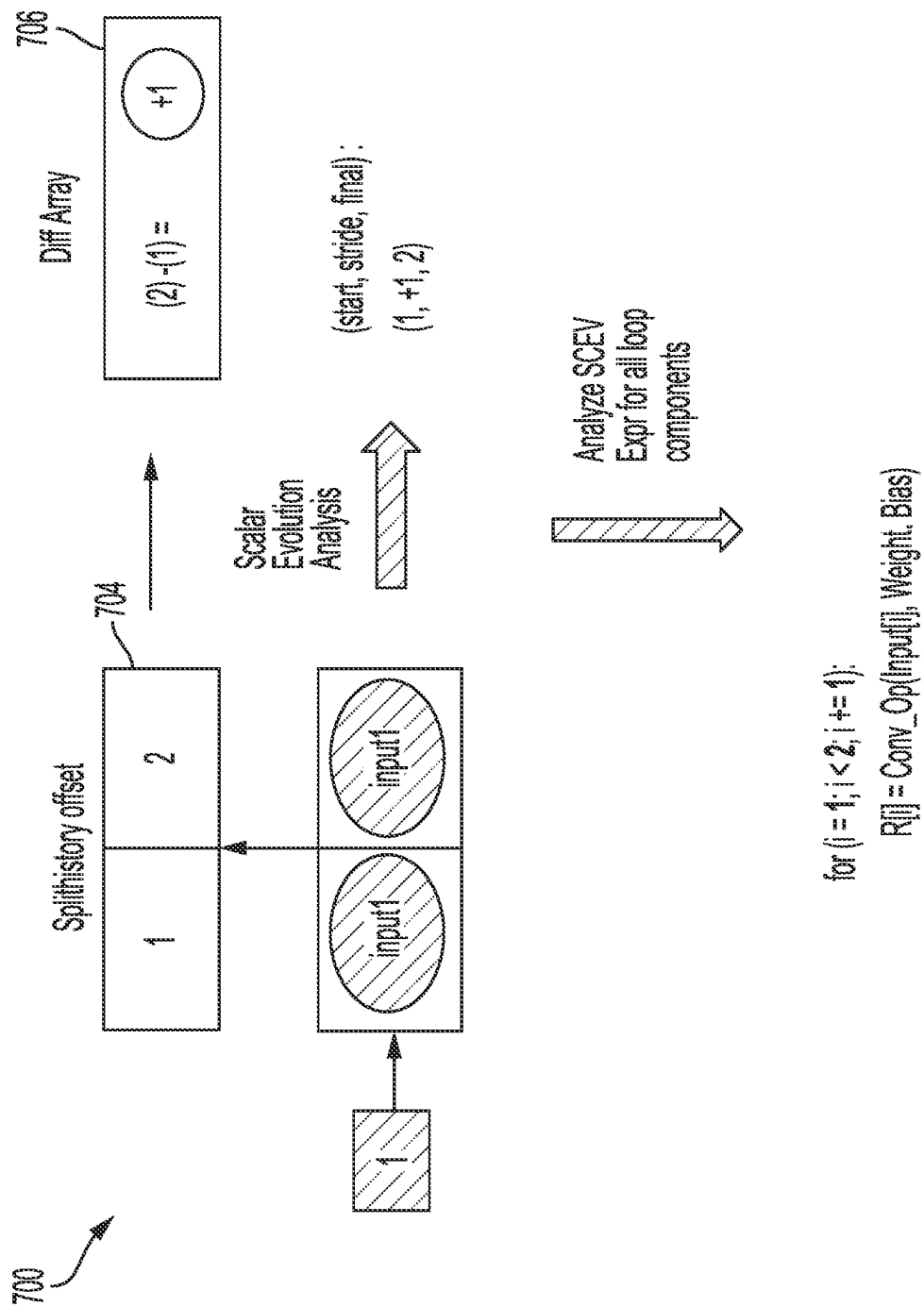
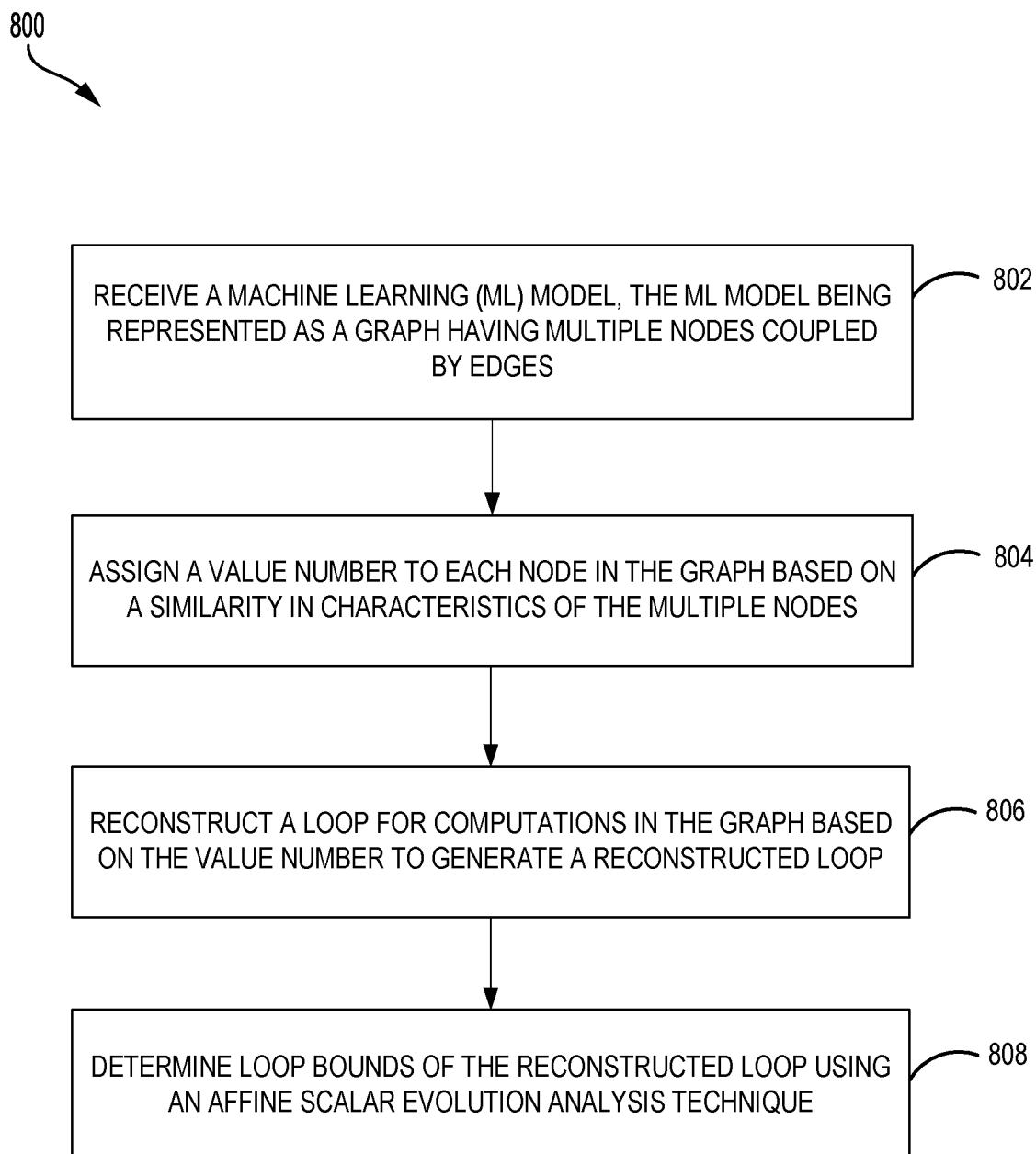


FIG. 7

**FIG. 8**

LOOP OPTIMIZATION-BASED APPROACH FOR TASK SCHEDULING IN GRAPH MACHINE LEARNING MODELS

FIELD OF THE DISCLOSURE

[0001] Aspects of the present disclosure generally relate to machine learning, and more particularly to scheduling tasks in graph machine learning models using loop optimizations.

BACKGROUND

[0002] Artificial neural networks may comprise interconnected groups of artificial neurons (e.g., neuron models). The artificial neural network (ANN) may be a computational device or be represented as a method to be performed by a computational device. Convolutional neural networks (CNNs) are a type of feed-forward ANN. Convolutional neural networks may include collections of neurons that each have a receptive field and that collectively tile an input space. Convolutional neural networks, such as deep convolutional neural networks (DCNs), have numerous applications. In particular, these neural network architectures are used in various technologies, such as image recognition, speech recognition, acoustic scene classification, keyword spotting, autonomous driving, and other classification tasks.

[0003] The tasks of a machine learning (ML) model may be scheduled, and data allocated by the ML optimization framework. Optimal graph scheduling under resource constraints may be a non-deterministic polynomial (NP)-complete problem, which may result in poor tractability of the ML optimizers as the width of the models increase. As a result, the time it takes for these optimizers to optimize an ML model may be unacceptable to a user.

SUMMARY

[0004] In some aspects of the present disclosure, a processor-implemented method includes receiving a machine learning (ML) model. The ML model is represented as a graph having multiple nodes coupled by edges. The processor-implemented method also includes assigning a value number to each node in the graph based on a similarity in characteristics of the multiple nodes. The processor-implemented method additionally includes reconstructing a loop for computations in the graph based on the value number to generate a reconstructed loop. The processor-implemented method further includes determining loop bounds of the reconstructed loop using an affine scalar evolution analysis technique.

[0005] Various aspects of the present disclosure are directed to an apparatus including means for receiving a machine learning (ML) model. The ML model is represented as a graph having multiple nodes coupled by edges. The apparatus also includes means for assigning a value number to each node in the graph based on a similarity in characteristics of the multiple nodes. The apparatus additionally includes means for reconstructing a loop for computations in the graph based on the value number to generate a reconstructed loop. The apparatus further includes means for determining loop bounds of the reconstructed loop using an affine scalar evolution analysis technique.

[0006] Some aspects of the present disclosure are directed to an apparatus having at least one memory and one or more processors coupled to the at least one memory. The processor(s) is configured to receive a machine learning (ML)

model. The ML model is represented as a graph having multiple nodes coupled by edges. The processor(s) is also configured to assign a value number to each node in the graph based on a similarity in characteristics of the multiple nodes. The processor(s) is additionally configured to reconstruct a loop for computations in the graph based on the value number to generate a reconstructed loop. The processor(s) is further configured to determine loop bounds of the reconstructed loop using an affine scalar evolution analysis technique.

[0007] Additional features and advantages of the disclosure will be described below. It should be appreciated by those skilled in the art that this disclosure may be readily utilized as a basis for modifying or designing other structures for carrying out the same purposes of the present disclosure. It should also be realized by those skilled in the art that such equivalent constructions do not depart from the teachings of the disclosure as set forth in the appended claims. The novel features, which are believed to be characteristic of the disclosure, both as to its organization and method of operation, together with further objects and advantages, will be better understood from the following description when considered in connection with the accompanying figures. It is to be expressly understood, however, that each of the figures is provided for the purpose of illustration and description only and is not intended as a definition of the limits of the present disclosure.

BRIEF DESCRIPTION OF THE DRAWINGS

[0008] The features, nature, and advantages of the present disclosure will become more apparent from the detailed description set forth below when taken in conjunction with the drawings in which like reference characters identify correspondingly throughout.

[0009] FIG. 1 illustrates an example implementation of a neural network using a system-on-a-chip (SOC), including a general-purpose processor, in accordance with certain aspects of the present disclosure.

[0010] FIGS. 2A, 2B, and 2C are diagrams illustrating a neural network, in accordance with various aspects of the present disclosure.

[0011] FIG. 2D is a diagram illustrating an exemplary deep convolutional network (DCN), in accordance with various aspects of the present disclosure.

[0012] FIG. 3 is a block diagram illustrating an exemplary deep convolutional network (DCN), in accordance with various aspects of the present disclosure.

[0013] FIG. 4 is a block diagram illustrating an exemplary software architecture that may modularize artificial intelligence (AI) functions, in accordance with various aspects of the present disclosure.

[0014] FIG. 5 is a block diagram illustrating an example value numbering process, in accordance with various aspects of the present disclosure.

[0015] FIG. 6 is a block diagram illustrating an example process for loop re-rolling, in accordance with various aspects of the present disclosure.

[0016] FIG. 7 is a block diagram illustrating an example scalar evolution analysis, in accordance with various aspects of the present disclosure.

[0017] FIG. 8 is a flow diagram illustrating a processor-implemented method for scheduling tasks in graph machine learning models using loop optimizations, in accordance with various aspects of the present disclosure.

DETAILED DESCRIPTION

[0018] The detailed description set forth below, in connection with the appended drawings, is intended as a description of various configurations and is not intended to represent the only configurations in which the concepts described may be practiced. The detailed description includes specific details for the purpose of providing a thorough understanding of the various concepts. However, it will be apparent to those skilled in the art that these concepts may be practiced without these specific details. In some instances, well-known structures and components are shown in block diagram form in order to avoid obscuring such concepts.

[0019] Based on the teachings, one skilled in the art should appreciate that the scope of the disclosure is intended to cover any aspect of the disclosure, whether implemented independently of or combined with any other aspect of the disclosure. For example, an apparatus may be implemented or a method may be practiced using any number of the aspects set forth. In addition, the scope of the disclosure is intended to cover such an apparatus or method practiced using other structure, functionality, or structure and functionality in addition to or other than the various aspects of the disclosure set forth. It should be understood that any aspect of the disclosure disclosed may be embodied by one or more elements of a claim.

[0020] The word “exemplary” is used to mean “serving as an example, instance, or illustration.” Any aspect described as “exemplary” is not necessarily to be construed as preferred or advantageous over other aspects.

[0021] Although particular aspects are described, many variations and permutations of these aspects fall within the scope of the disclosure. Although some benefits and advantages of the preferred aspects are mentioned, the scope of the disclosure is not intended to be limited to particular benefits, uses or objectives. Rather, aspects of the disclosure are intended to be broadly applicable to different technologies, system configurations, networks, and protocols, some of which are illustrated by way of example in the figures and in the following description of the preferred aspects. The detailed description and drawings are merely illustrative of the disclosure rather than limiting, the scope of the disclosure being defined by the appended claims and equivalents thereof.

[0022] Many machine learning (ML) optimization tools (e.g., compilers) represent the computations of the ML model as a graph of tasks, in which vertices represent computations, and edges represent data dependencies among the computations. The compilers may utilize an unrolled representation of the ML model to perform such optimizations. In some aspects, the unrolled representation may be unknowingly produced. That is, an ML author may not realize that the process performed by the ML model corresponds to unrolling. For example, a generative pre-trained transformer (GPT) model may include a sequence of transformers, which may produce unrolled loops.

[0023] The graph may comprise a directed acyclic graph of layers, for instance. The layers may be tiled and may represent a task. The optimizations may include scheduling execution of tasks and/or memory allocation, for instance. However, the size of the graph increases as the width of the model increases. The size of the inputs may have direct impact on the number of nodes in the graph. This is because tile sizes may be fixed (e.g., constant). The tensors of the ML

model may be two-dimensional (2D) or higher, and thus, the size of the graph may increase at a rate of $O(n^2)$.

[0024] Each node may represent a task in the graph. The tasks may be scheduled, and data allocated by the ML optimization framework. Optimal graph scheduling under resource constraints may be a non-deterministic polynomial time (NP)-complete (NP-complete) problem, which may result in poor tractability of the ML optimizers as the width of the models increase. As a result, the time it takes for these optimizers to optimize an ML model may be unacceptable to a user.

[0025] To address these and other problems, scheduling of larger task graphs may leverage the highly repetitive nature of such graphs to convert the graphs to a loop representation. That is, the graph may be reconstructed in the form of loops. Each task becomes an iteration of a loop-based program. The loops may access data elements, which may be described as a function (e.g., linear or non-linear) of the loop counter, so the tasks and the data consumed and produced may be indexed and represented as a loop counter (then, loop optimization techniques may be applied to determine a schedule for the tasks).

[0026] Particular aspects of the subject matter described in this disclosure can be implemented to realize one or more of the following potential advantages. In some examples, the described techniques (e.g., assigning value numbers to similar substructures of the graph, reconstructing loops based on the value numbers, and performing scalar evolution analysis) may beneficially reduce latency in computing a schedule for executing nodes of the ML model. This is because the time to compute a schedule is constant as a function of the model width.

[0027] In addition, by representing the ML computations using loops, various loop optimization techniques may be applied to improve scheduling performance and ML model processing efficiency. For instance, such loop optimizations may include loop fusion, loop tiling, placement across multiple cores, direct memory access (DMA) generation and coarsening, multi-buffering (sometimes referred to as high-level software pipelining), and other loop optimizations.

[0028] Moreover, performance debugging may also be simplified because the behavior of many tasks (of the same loop) may be analyzed at once, as opposed to analyzing the sum of their individual behavior, as is the case in some conventional approaches.

[0029] Furthermore, optimization decisions may be performed in bulk over the original tasks, making for a more regular behavior. Regularity may enable allocation of the same local memory offsets for a certain piece of data on all the cores. A multicast transmission to the same address everywhere may be more efficient than allocation of the same address to a different address on each core.

[0030] FIG. 1 illustrates an example implementation of a system-on-a-chip (SOC) 100, which may include a central processing unit (CPU) 102 or a multi-core CPU configured for scheduling tasks in graph machine learning models using loop optimizations. Variables (e.g., neural signals and synaptic weights), system parameters associated with a computational device (e.g., neural network with weights), delays, frequency bin information, and task information may be stored in a memory block associated with a neural processing unit (NPU) 108, in a memory block associated with a CPU 102, in a memory block associated with a graphics processing unit (GPU) 104, in a memory block

associated with a digital signal processor (DSP) **106**, in a memory block **118**, or may be distributed across multiple blocks. Instructions executed at the CPU **102** may be loaded from a program memory associated with the CPU **102** or may be loaded from a memory block **118**.

[0031] The SOC **100** may also include additional processing blocks tailored to specific functions, such as a GPU **104**, a DSP **106**, a connectivity block **110**, which may include fifth generation (5G) connectivity, fourth generation long term evolution (4G LTE) connectivity, Wi-Fi connectivity, USB connectivity, Bluetooth connectivity, and the like, and a multimedia processor **112** that may, for example, detect and recognize gestures. In one implementation, the NPU **108** is implemented in the CPU **102**, DSP **106**, and/or GPU **104**. The SOC **100** may also include a sensor processor **114**, image signal processors (ISPs) **116**, and/or navigation module **120**, which may include a global positioning system.

[0032] The SOC **100** may be based on an ARM, RISC-V (RISC-five), or any reduced instruction set computing (RISC) architecture. In aspects of the present disclosure, the instructions loaded into the general-purpose processor **102** may include code to receive a machine learning (ML) model. The ML model is represented as a graph having multiple nodes coupled by edges. The instructions loaded into the general-purpose processor **102** may also include code to assign a value number to each node in the graph based on a similarity in characteristics of the multiple nodes. The instructions loaded into the general-purpose processor **102** may additionally include code to reconstruct a loop for computations in the graph based on the value number to generate a reconstructed loop. The instructions loaded into the general-purpose processor **102** may further include code to determine loop bounds of the reconstructed loop using an affine scalar evolution analysis technique.

[0033] Deep learning architectures may perform an object recognition task by learning to represent inputs at successively higher levels of abstraction in each layer, thereby building up a useful feature representation of the input data. In this way, deep learning addresses a major bottleneck of traditional machine learning. Prior to the advent of deep learning, a machine learning approach to an object recognition problem may have relied heavily on human engineered features, perhaps in combination with a shallow classifier. A shallow classifier may be a two-class linear classifier, for example, in which a weighted sum of the feature vector components may be compared with a threshold to predict to which class the input belongs. Human engineered features may be templates or kernels tailored to a specific problem domain by engineers with domain expertise. Deep learning architectures, in contrast, may learn to represent features that are similar to what a human engineer might design, but through training. Furthermore, a deep network may learn to represent and recognize new types of features that a human might not have considered.

[0034] A deep learning architecture may learn a hierarchy of features. If presented with visual data, for example, the first layer may learn to recognize relatively simple features, such as edges, in the input stream. In another example, if presented with auditory data, the first layer may learn to recognize spectral power in specific frequencies. The second layer, taking the output of the first layer as input, may learn to recognize combinations of features, such as simple shapes for visual data or combinations of sounds for auditory data. For instance, higher layers may learn to represent complex

shapes in visual data or words in auditory data. Still higher layers may learn to recognize common visual objects or spoken phrases.

[0035] Deep learning architectures may perform especially well when applied to problems that have a natural hierarchical structure. For example, the classification of motorized vehicles may benefit from first learning to recognize wheels, windshields, and other features. These features may be combined at higher layers in different ways to recognize cars, trucks, and airplanes.

[0036] Neural networks may be designed with a variety of connectivity patterns. In feed-forward networks, information is passed from lower to higher layers, with each neuron in a given layer communicating to neurons in higher layers. A hierarchical representation may be built up in successive layers of a feed-forward network, as described above. Neural networks may also have recurrent or feedback (also called top-down) connections. In a recurrent connection, the output from a neuron in a given layer may be communicated to another neuron in the same layer. A recurrent architecture may be helpful in recognizing patterns that span more than one of the input data chunks that are delivered to the neural network in a sequence. A connection from a neuron in a given layer to a neuron in a lower layer is called a feedback (or top-down) connection. A network with many feedback connections may be helpful when the recognition of a high-level concept may aid in discriminating the particular low-level features of an input.

[0037] The connections between layers of a neural network may be fully connected or locally connected. FIG. 2A illustrates an example of a fully connected neural network **202**. In a fully connected neural network **202**, a neuron in a first layer may communicate its output to every neuron in a second layer, so that each neuron in the second layer will receive input from every neuron in the first layer. FIG. 2B illustrates an example of a locally connected neural network **204**. In a locally connected neural network **204**, a neuron in a first layer may be connected to a limited number of neurons in the second layer. More generally, a locally connected layer of the locally connected neural network **204** may be configured so that each neuron in a layer will have the same or a similar connectivity pattern, but with connections strengths that may have different values (e.g., **210**, **212**, **214**, and **216**). The locally connected connectivity pattern may give rise to spatially distinct receptive fields in a higher layer because the higher layer neurons in a given region may receive inputs that are tuned through training to the properties of a restricted portion of the total input to the network.

[0038] One example of a locally connected neural network is a convolutional neural network. FIG. 2C illustrates an example of a convolutional neural network **206**. The convolutional neural network **206** may be configured such that the connection strengths associated with the inputs for each neuron in the second layer are shared (e.g., **208**). Convolutional neural networks may be well suited to problems in which the spatial location of inputs is meaningful.

[0039] One type of convolutional neural network is a deep convolutional network (DCN). FIG. 2D illustrates a detailed example of a DCN **200** designed to recognize visual features from an image **226** input from an image capturing device **230**, such as a car-mounted camera. The DCN **200** of the current example may be trained to identify traffic signs and a number provided on the traffic sign. Of course, the DCN

200 may be trained for other tasks, such as identifying lane markings or identifying traffic lights.

[0040] The DCN **200** may be trained with supervised learning. During training, the DCN **200** may be presented with an image, such as the image **226** of a speed limit sign, and a forward pass may then be computed to produce an output **222**. The DCN **200** may include a feature extraction section and a classification section. Upon receiving the image **226**, a convolutional layer **232** may apply convolutional kernels (not shown) to the image **226** to generate a first set of feature maps **218**. As an example, the convolutional kernel for the convolutional layer **232** may be a 5×5 kernel that generates 28×28 feature maps. In the present example, because four different feature maps are generated in the first set of feature maps **218**, four different convolutional kernels were applied to the image **226** at the convolutional layer **232**. The convolutional kernels may also be referred to as filters or convolutional filters.

[0041] The first set of feature maps **218** may be sub-sampled by a max pooling layer (not shown) to generate a second set of feature maps **220**. The max pooling layer reduces the size of the first set of feature maps **218**. That is, a size of the second set of feature maps **220**, such as 14×14, is less than the size of the first set of feature maps **218**, such as 28×28. The reduced size provides similar information to a subsequent layer while reducing memory consumption. The second set of feature maps **220** may be further convolved via one or more subsequent convolutional layers (not shown) to generate one or more subsequent sets of feature maps (not shown).

[0042] In the example of FIG. 2D, the second set of feature maps **220** is convolved to generate a first feature vector **224**. Furthermore, the first feature vector **224** is further convolved to generate a second feature vector **228**. Each feature of the second feature vector **228** may include a number that corresponds to a possible feature of the image **226**, such as “sign,” “60,” and “100.” A softmax function (not shown) may convert the numbers in the second feature vector **228** to a probability. As such, an output **222** of the DCN **200** may be a probability of the image **226** including one or more features.

[0043] In the present example, the probabilities in the output **222** for “sign” and “60” are higher than the probabilities of the others of the output **222**, such as “30,” “40,” “50,” “70,” “80,” “90,” and “100.” Before training, the output **222** produced by the DCN **200** may likely be incorrect. Thus, an error may be calculated between the output **222** and a target output. The target output is the ground truth of the image **226** (e.g., “sign” and “60”). The weights of the DCN **200** may then be adjusted so the output **222** of the DCN **200** is more closely aligned with the target output.

[0044] To adjust the weights, a learning algorithm may compute a gradient vector for the weights. The gradient may indicate an amount that an error would increase or decrease if the weight were adjusted. At the top layer, the gradient may correspond directly to the value of a weight connecting an activated neuron in the penultimate layer and a neuron in the output layer. In lower layers, the gradient may depend on the value of the weights and on the computed error gradients of the higher layers. The weights may then be adjusted to reduce the error. This manner of adjusting the weights may be referred to as “back propagation” as it involves a “backward pass” through the neural network.

[0045] In practice, the error gradient of weights may be calculated over a small number of examples, so that the calculated gradient approximates the true error gradient. This approximation method may be referred to as stochastic gradient descent. Stochastic gradient descent may be repeated until the achievable error rate of the entire system has stopped decreasing or until the error rate has reached a target level. After learning, the DCN **200** may be presented with new images (e.g., the speed limit sign of the image **226**) and a forward pass through the DCN **200** may yield an output **222** that may be considered an inference or a prediction of the DCN **200**.

[0046] Deep belief networks (DBNs) are probabilistic models comprising multiple layers of hidden nodes. DBNs may be used to extract a hierarchical representation of training data sets. A DBN may be obtained by stacking up layers of Restricted Boltzmann Machines (RBMs). An RBM is a type of artificial neural network that can learn a probability distribution over a set of inputs. Because RBMs can learn a probability distribution in the absence of information about the class to which each input should be categorized, RBMs are often used in unsupervised learning. Using a hybrid unsupervised and supervised paradigm, the bottom RBMs of a DBN may be trained in an unsupervised manner and may serve as feature extractors, and the top RBM may be trained in a supervised manner (on a joint distribution of inputs from the previous layer and target classes) and may serve as a classifier.

[0047] DCNs are networks of convolutional networks, configured with additional pooling and normalization layers. DCNs have achieved state-of-the-art performance on many tasks. DCNs can be trained using supervised learning in which both the input and output targets are known for many exemplars and are used to modify the weights of the network by use of gradient descent methods.

[0048] DCNs may be feed-forward networks. In addition, as described above, the connections from a neuron in a first layer of a DCN to a group of neurons in the next higher layer are shared across the neurons in the first layer. The feed-forward and shared connections of DCNs may be exploited for fast processing. The computational burden of a DCN may be much less, for example, than that of a similarly sized neural network that comprises recurrent or feedback connections.

[0049] The processing of each layer of a convolutional network may be considered a spatially invariant template or basis projection. If the input is first decomposed into multiple channels, such as the red, green, and blue channels of a color image, then the convolutional network trained on that input may be considered three-dimensional, with two spatial dimensions along the axes of the image and a third dimension capturing color information. The outputs of the convolutional connections may be considered to form a feature map in the subsequent layer, with each element of the feature map (e.g., **220**) receiving input from a range of neurons in the previous layer (e.g., feature maps **218**) and from each of the multiple channels. The values in the feature map may be further processed with a non-linearity, such as a rectification, $\max(0, x)$. Values from adjacent neurons may be further pooled, which corresponds to down sampling, and may provide additional local invariance and dimensionality reduction. Normalization, which corresponds to whitening, may also be applied through lateral inhibition between neurons in the feature map.

[0050] FIG. 3 is a block diagram illustrating a DCN 350. The DCN 350 may include multiple different types of layers based on connectivity and weight sharing. As shown in FIG. 3, the DCN 350 includes the convolution blocks 354A, 354B. Each of the convolution blocks 354A, 354B may be configured with a convolution layer (CONV) 356, a normalization layer (LNorm) 358, and a max pooling layer (MAX POOL) 360.

[0051] Although only two of the convolution blocks 354A, 354B are shown, the present disclosure is not so limiting, and instead, any number of the convolution blocks 354A, 354B may be included in the DCN 350 according to design preference.

[0052] The convolution layers 356 may include one or more convolutional filters, which may be applied to the input data to generate a feature map. The normalization layer 358 may normalize the output of the convolution filters. For example, the normalization layer 358 may provide whitening or lateral inhibition. The max pooling layer 360 may provide down sampling aggregation over space for local invariance and dimensionality reduction.

[0053] The parallel filter banks, for example, of a deep convolutional network may be loaded on a CPU 102 or GPU 104 of an SOC 100 (e.g., FIG. 1) to achieve high performance and low power consumption. In alternative embodiments, the parallel filter banks may be loaded on the DSP 106 or an ISP 116 of an SOC 100. In addition, the DCN 350 may access other processing blocks that may be present on the SOC 100, such as sensor processor 114 and navigation module 120, dedicated, respectively, to sensors and navigation.

[0054] The DCN 350 may also include one or more fully connected layers 362 (FC1 and FC2). The DCN 350 may further include a logistic regression (LR) layer 364. Between each layer 356, 358, 360, 362, 364 of the DCN 350 are weights (not shown) that are to be updated. The output of each of the layers (e.g., 356, 358, 360, 362, 364) may serve as an input of a succeeding one of the layers (e.g., 356, 358, 360, 362, 364) in the DCN 350 to learn hierarchical feature representations from input data 352 (e.g., images, audio, video, sensor data and/or other input data) supplied at the first of the convolution blocks 354A. The output of the DCN 350 is a classification score 366 for the input data 352. The classification score 366 may be a set of probabilities, where each probability is the probability of the input data including a feature from a set of features.

[0055] FIG. 4 is a block diagram illustrating an exemplary software architecture 400 that may modularize artificial intelligence (AI) functions. Using the architecture 400, applications may be designed that may cause various processing blocks of an SOC 420 (for example a CPU 422, a DSP 424, a GPU 426 and/or an NPU 428) (which may be similar to SOC 100 of FIG. 1) to support loop optimization-based task scheduling for an AI application 402, according to aspects of the present disclosure. The architecture 400 may, for example, be included in a computational device, such as a smartphone.

[0056] The AI application 402 may be configured to call functions defined in a user space 404 that may, for example, provide for the detection and recognition of a scene indicative of the location at which the computational device including the architecture 400 currently operates. The AI application 402 may, for example, configure a microphone and a camera differently depending on whether the recog-

nized scene is an office, a lecture hall, a restaurant, or an outdoor setting such as a lake. The AI application 402 may make a request to compiled program code associated with a library defined in an AI function application programming interface (API) 406. This request may ultimately rely on the output of a deep neural network configured to provide an inference response based on video and positioning data, for example.

[0057] The run-time engine 408, which may be compiled code of a runtime framework, may be further accessible to the AI application 402. The AI application 402 may cause the run-time engine 408, for example, to request an inference at a particular time interval or triggered by an event detected by the user interface of the AI application 402. When caused to provide an inference response, the run-time engine 408 may in turn send a signal to an operating system in an operating system (OS) space 410, such as a Kernel 412, running on the SOC 420. In some examples, the Kernel 412 may be a LINUX Kernel. The operating system, in turn, may cause a continuous relaxation of quantization to be performed on the CPU 422, the DSP 424, the GPU 426, the NPU 428, or some combination thereof. The CPU 422 may be accessed directly by the operating system, and other processing blocks may be accessed through a driver, such as a driver 414, 416, or 418 for, respectively, the DSP 424, the GPU 426, or the NPU 428. In the exemplary example, the deep neural network may be configured to run on a combination of processing blocks, such as the CPU 422, the DSP 424, and the GPU 426, or may be run on the NPU 428.

[0058] FIG. 5 is a block diagram illustrating an example value numbering process 500, in accordance with various aspects of the present disclosure. Referring to FIG. 5, the example value numbering process 500 may receive a graph 502 as input. The graph 502 may represent a ML model. The ML model may be (but is not limited to) a transformer model, for example. The graph 502 may be (but is not limited to) a directed acyclic graph (DAG), for example. The graph 502 includes multiple nodes connected by edges. The graph 502 includes input nodes (e.g., input1 and input2), a weight node, a bias node, and convolution nodes (e.g., Conv1 and Conv2). The number and types of nodes shown are merely an example for ease of illustration and understanding, and not limiting. Rather, graph 502 may include any number of nodes and different types of nodes.

[0059] The value numbering process 500 may conduct a value numbering pass 506. In the value numbering pass 506, the value numbering process 500 may assign a value number to each of the inputs, nodes, and constants. The value numbering process 500 may use similar characteristics (e.g., single static assignments (SSA) and intermediate representation (IR)) as well as control flow to group the nodes. In various aspects, input nodes may be assigned constant value numbers by hashing according to an operation name or a group identifier, for instance. For example, the input nodes (e.g., input1 and input2) may be grouped because of the similarity in node type. The input nodes may be assigned value number 1. The other nodes may be assigned by hashing on operation name, number of operands, for example.

[0060] Accordingly, as shown in FIG. 5, the weight node and bias node have a different node type than other nodes in the graph 502, and may be respectively assigned value numbers 2 and 3. Finally, the convolution nodes (Conv1 and Conv2), have a same node type and both consume an input

(e.g., input1 or input2) as well as the weight node and the bias node. Thus, the convolution nodes (e.g., Conv1 and Conv2) may be grouped and assigned a value number 4. In various aspects, each of the groups (may also be referred to as buckets) may represent identical computation expressions. An analysis of the node similarity may be complexity bounded by a number of edges.

[0061] FIG. 6 is a block diagram illustrating an example process 600 for loop re-rolling, in accordance with various aspects of the present disclosure. Re-rolling may refer to reconstructing loops and loop access based on edges in the graph (e.g., 502 of FIG. 5). The example process 600 for loop re-rolling may conduct loop re-rolling 602 to reconstruct loops and loop accesses. In various aspects, the identical computation expressions may indicate an individual iteration of an unrolled loop. For example,

$R1 = \text{Conv_Op}(\text{Input1}, \text{Weight}, \text{Bias})$

$R2 = \text{Conv_Op}(\text{Input2}, \text{Weight}, \text{Bias}),$

may be expressed as:

for ($i=0, I<2; i++$)

$R[i] = \text{Conv_Op}(\text{Input}[i], \text{Weight}, \text{Bias}).$

[0062] A new DAG 604 may be generated. The nodes of the DAG 604 may be identified using valued numbers. Each valued number may represent an operation in a loop body. For example, node 1 may represent the input nodes (e.g., input1 and input2 from graph 502 of FIG. 5), nodes 2 and 3 may respectively represent the weight node and the bias node, and node 4 may represent the convolution nodes (e.g., Conv1 and Conv2).

[0063] Information about loop bounds, access functions, and number of loop variables/loop nests within each loop body, however, may be unknown. For instance, the start iteration, final iteration, and stride may be unknown, as follows:

for ($i=\text{start}??; i<\text{final}??; i+=\text{stride}??$):

$R[i] = \text{Conv_Op}(\text{Input}[i], \text{Weight}, \text{Bias})$

[0064] To determine these loop characteristics (e.g., start iteration, final iteration, or stride), affine expressions that define each induction variable/derived induction variables in the loop may be determined, for example, using scalar evolution analysis.

[0065] FIG. 7 is a block diagram illustrating an example scalar evolution analysis 700, in accordance with various aspects of the present disclosure. Scalar evolution analysis refers to a compiler technique that involves an analysis for deriving the stride and upper and lower bounds of a loop. A split history of each operation in a graph may provide the offset of each node (e.g., operation) value across the tiled dimensions. The offsets for each node in each dimension may be mapped to an iteration sequence. The stride of the loop may be derived by taking the difference between neighboring offsets.

[0066] Referring to FIG. 7, a split history offset 704 corresponding to input nodes (e.g., input1, input2) may be determined. A difference array 706 may store the result of a difference between the split history offsets (e.g., (2)-(1)=+1). Accordingly, the stride of the loop may be +1.

[0067] A first element in the array (of the split history offset 704) may represent the start value and the last element

may represent the final value. Accordingly, the start value may be determined to be 1 and the final value may be determined to be 2.

[0068] The affine scalar evolution (SCEV) may be derived for the operands and the result of the loop using their respective split history (e.g., 704). The SCEV of all components may aid in deriving the following: (a) loop variables based on a number of dimensions, (b) an iteration length across each dimension (final-start), (c) consistency across all SCEV expressions to define the loop control construct, and (d) the SCEV expression, which may also define an access function of each node in the loop. Accordingly, the re-rolled loop expression may be expressed as follows:

for ($i=1; i<2; i+=1$):

$R[i] = \text{Conv_Op}(\text{Input}[i], \text{Weight}, \text{Bias})$

[0069] FIG. 8 is a flow diagram illustrating a processor-implemented method 800 for scheduling tasks in graph machine learning models using loop optimizations, in accordance with various aspects of the present disclosure. The processor-implemented method 800 may be performed by one or more processors such as the CPU (e.g., 102, 422), GPU (e.g., 104, 426), and/or other processing unit (e.g., DSP 424, NPU 428), for example. In some aspects, the processor-implemented method may be performed in a compiler, for instance.

[0070] At block 802, the at least one processor receives a machine learning (ML) model, the ML model being represented as a graph having multiple nodes coupled by edges. As described, for example, with reference to FIG. 5, the example value numbering process 500 may receive a graph 502 as input. The graph 502 may represent a ML model. The ML model may be (but is not limited to) a transformer model, for example. The graph 502 may be (but is not limited to) a directed acyclic graph (DAG), for example. The graph 502 may include multiple nodes connected by edges. The graph 502 may include input nodes (e.g., input1 and input2), a weight node, a bias node, and convolution nodes (e.g., Conv1 and Conv2).

[0071] At block 804, the at least one processor assigns a value number to each node in the graph based on a similarity in characteristics of the multiple nodes. For instance, as described with reference to FIG. 5, the value numbering process 500 may conduct a value numbering pass 506. In the value numbering pass 506, the value numbering process 500 may assign a value number to each of the inputs, nodes, and constants. The value numbering process 500 may use similar characteristics (e.g., single static assignments (SSA) and intermediate representation (IR)) as well as control flow to group the nodes. In various aspects, input nodes may be assigned constant value numbers by hashing according to an operation name or a group identifier, for instance. For example, the input nodes (e.g., input1 and input2) may be grouped because of the similarity in node type. The input nodes may be assigned value number 1. The other nodes may be assigned by hashing on operation name, number of operands, for example.

[0072] At block 806, the at least one processor reconstructs a loop for computations in the graph based on the value number to generate a reconstructed loop. For example, as described with reference to FIG. 6, loops may be reconstructed (may also be referred to as re-rolled) based on edges in the graph (e.g., 502 of FIG. 5). The example process 600 for loop re-rolling may conduct loop re-rolling 602 to

reconstruct loops and loop accesses. In various aspects, the identical computation expressions may indicate an individual iteration of an unrolled loop.

[0073] At block **808**, the at least one processor determine loop bounds of the reconstructed loop using an affine scalar evolution analysis technique. As described, for instance, with reference to FIG. 7, scalar evolution analysis refers to a compiler technique that involves an analysis for deriving the stride and upper and lower bounds of a loop. The affine scalar evolution (SCEV) may be derived for the operands and the result of the loop using the respective split history (e.g., **704**) of the operands. A split history of each operation in a graph may provide the offset of each node (e.g., operation) value across the tiled dimensions. The offsets for each node in each dimension may be mapped to an iteration sequence. The stride of the loop may be derived by taking the difference between neighboring offsets.

[0074] The SCEV of all components may aid in deriving the following: (a) loop variables based on a number of dimensions, (b) an iteration length across each dimension (final-start), (c) consistency across all SCEV expressions to define the loop control construct, and (d) the SCEV expression, which may also define an access function of each node in the loop.

EXAMPLE ASPECTS

[0075] Aspect 1: An apparatus comprising: at least one memory; and at least one processor coupled to the at least one memory, the at least one processor configured to: receive a machine learning (ML) model, the ML model being represented as a graph having multiple nodes coupled by edges; assign a value number to each node in the graph based on a similarity in characteristics of the multiple nodes; reconstruct a loop for computations in the graph based on the value number to generate a reconstructed loop; and determine loop bounds of the reconstructed loop using an affine scalar evolution analysis technique.

[0076] Aspect 2: The apparatus of Aspect 1, in which the at least one processor is further configured to group the multiple nodes into a group by applying a hashing function based on a node type.

[0077] Aspect 3: The apparatus of Aspect 1 or 2, in which the group represents identical computation expressions.

[0078] Aspect 4: The apparatus of any preceding Aspects, in which the identical computation expressions correspond to iterations of an unrolled loop.

[0079] Aspect 5: The apparatus of any preceding Aspects, in which the loop bounds are determined based on a split history offset of each operation in the graph.

[0080] Aspect 6: The apparatus of any preceding Aspects, in which the at least one processor is further configured to determine a stride of the reconstructed loop based on a difference between neighboring split history offsets.

[0081] Aspect 7: The apparatus of any preceding Aspects, in which the at least one processor is further configured to determine a schedule for performing operations in the ML model using one or more loop optimization techniques on the reconstructed loop.

[0082] Aspect 8: A processor-implemented method performed by one or more processors, the processor-

implemented method comprising: receiving a machine learning (ML) model, the ML model being represented as a graph having multiple nodes coupled by edges; assigning a value number to each node in the graph based on a similarity in characteristics of the multiple nodes; reconstructing a loop for computations in the graph based on the value number to generate a reconstructed loop; and determining loop bounds of the reconstructed loop using an affine scalar evolution analysis technique.

[0083] Aspect 9: The processor-implemented method of Aspect 8, further comprising grouping the multiple nodes into a group by applying a hashing function based on a node type.

[0084] Aspect 10: The processor-implemented method of Aspect 8 or 9, in which the group represents identical computation expressions.

[0085] Aspect 11: The processor-implemented method of any of Aspects 8-10, in which the identical computation expressions correspond to iterations of an unrolled loop.

[0086] Aspect 12: The processor-implemented method of any of Aspects 8-11, in which the loop bounds are determined based on a split history offset of each operation in the graph.

[0087] Aspect 13: The processor-implemented method of any of Aspects 8-12, further comprising determining a stride of the reconstructed loop based on a difference between neighboring split history offsets.

[0088] Aspect 14: The processor-implemented method of any of Aspects 8-13, further comprising determining a schedule for performing operations in the ML model using one or more loop optimization techniques on the reconstructed loop.

[0089] Aspect 15: An apparatus comprising: means for receiving a machine learning (ML) model, the ML model being represented as a graph having multiple nodes coupled by edges; means for assigning a value number to each node in the graph based on a similarity in characteristics of the multiple nodes; means for reconstructing a loop for computations in the graph based on the value number to generate a reconstructed loop; and means for determining loop bounds of the reconstructed loop using an affine scalar evolution analysis technique.

[0090] Aspect 16: The apparatus of Aspect 15, further comprising means for grouping the multiple nodes into a group by applying a hashing function based on a node type.

[0091] Aspect 17: The apparatus of Aspect 15 or 16, in which the group represents identical computation expressions and the identical computation expressions correspond to iterations of an unrolled loop.

[0092] Aspect 18: The apparatus of any of any of Aspects 15-17, in which the loop bounds are determined based on a split history offset of each operation in the graph.

[0093] Aspect 19: The apparatus of any of Aspects 15-18, further comprising means for determining a stride of the reconstructed loop based on a difference between neighboring split history offsets.

[0094] Aspect 20: The apparatus of any of Aspects 15-19, further comprising means for determining a

schedule for performing operations in the ML model using one or more loop optimization techniques on the reconstructed loop.

[0095] In one aspect, the receiving means, assigning means, reconstructing means and/or determining means may be the CPU **102**, program memory associated with the CPU **102**, NPU **108**, the dedicated memory block **118**, fully connected layers **362**, NPU **428** and/or the routing connection processing unit **216** configured to perform the functions recited. In another configuration, the aforementioned means may be any module or any apparatus configured to perform the functions recited by the aforementioned means.

[0096] The various operations of methods described above may be performed by any suitable means capable of performing the corresponding functions. The means may include various hardware and/or software component(s) and/or module(s), including, but not limited to, a circuit, an application specific integrated circuit (ASIC), or processor. Generally, where there are operations illustrated in the figures, those operations may have corresponding counterpart means-plus-function components with similar numbering.

[0097] As used, the term “determining” encompasses a wide variety of actions. For example, “determining” may include calculating, computing, processing, deriving, investigating, looking up (e.g., looking up in a table, a database, or another data structure), ascertaining and the like. Additionally, “determining” may include receiving (e.g., receiving information), accessing (e.g., accessing data in a memory) and the like. Furthermore, “determining” may include resolving, selecting, choosing, establishing, and the like.

[0098] As used, a phrase referring to “at least one of” a list of items refers to any combination of those items, including single members. As an example, “at least one of: a, b, or c” is intended to cover: a, b, c, a-b, a-c, b-c, and a-b-c.

[0099] The various illustrative logical blocks, modules and circuits described in connection with the present disclosure may be implemented or performed with a general-purpose processor, a digital signal processor (DSP), an application specific integrated circuit (ASIC), a field programmable gate array signal (FPGA) or other programmable logic device (PLD), discrete gate or transistor logic, discrete hardware components or any combination thereof designed to perform the functions described. A general-purpose processor may be a microprocessor, but in the alternative, the processor may be any commercially available processor, controller, microcontroller, or state machine. A processor may also be implemented as a combination of computing devices, e.g., a combination of a DSP and a microprocessor, a plurality of microprocessors, one or more microprocessors in conjunction with a DSP core, or any other such configuration.

[0100] The steps of a method or algorithm described in connection with the present disclosure may be embodied directly in hardware, in a software module executed by a processor, or in a combination of the two. A software module may reside in any form of storage medium that is known in the art. Some examples of storage media that may be used include random access memory (RAM), read only memory (ROM), flash memory, erasable programmable read-only memory (EPROM), electrically erasable programmable read-only memory (EEPROM), registers, a hard disk, a removable disk, a CD-ROM and so forth. A software module

may comprise a single instruction, or many instructions, and may be distributed over several different code segments, among different programs, and across multiple storage media. A storage medium may be coupled to a processor such that the processor can read information from, and write information to, the storage medium. In the alternative, the storage medium may be integral to the processor.

[0101] The methods disclosed comprise one or more steps or actions for achieving the described method. The method steps and/or actions may be interchanged with one another without departing from the scope of the claims. In other words, unless a specific order of steps or actions is specified, the order and/or use of specific steps and/or actions may be modified without departing from the scope of the claims.

[0102] The functions described may be implemented in hardware, software, firmware, or any combination thereof. If implemented in hardware, an example hardware configuration may comprise a processing system in a device. The processing system may be implemented with a bus architecture. The bus may include any number of interconnecting buses and bridges depending on the specific application of the processing system and the overall design constraints. The bus may link together various circuits including a processor, machine-readable media, and a bus interface. The bus interface may be used to connect a network adapter, among other things, to the processing system via the bus. The network adapter may be used to implement signal processing functions. For certain aspects, a user interface (e.g., keypad, display, mouse, joystick, etc.) may also be connected to the bus. The bus may also link various other circuits such as timing sources, peripherals, voltage regulators, power management circuits, and the like, which are well known in the art, and therefore, will not be described any further.

[0103] The processor may be responsible for managing the bus and general processing, including the execution of software stored on the machine-readable media. The processor may be implemented with one or more general-purpose and/or special-purpose processors. Examples include microprocessors, microcontrollers, DSP processors, and other circuitry that can execute software. Software shall be construed broadly to mean instructions, data, or any combination thereof, whether referred to as software, firmware, middleware, microcode, hardware description language, or otherwise. Machine-readable media may include, by way of example, random access memory (RAM), flash memory, read only memory (ROM), programmable read-only memory (PROM), erasable programmable read-only memory (EPROM), electrically erasable programmable Read-only memory (EEPROM), registers, magnetic disks, optical disks, hard drives, or any other suitable storage medium, or any combination thereof. The machine-readable media may be embodied in a computer-program product. The computer-program product may comprise packaging materials.

[0104] In a hardware implementation, the machine-readable media may be part of the processing system separate from the processor. However, as those skilled in the art will readily appreciate, the machine-readable media, or any portion thereof, may be external to the processing system. By way of example, the machine-readable media may include a transmission line, a carrier wave modulated by data, and/or a computer product separate from the device, all which may be accessed by the processor through the bus

interface. Alternatively, or in addition, the machine-readable media, or any portion thereof, may be integrated into the processor, such as the case may be with cache and/or general register files. Although the various components discussed may be described as having a specific location, such as a local component, they may also be configured in various ways, such as certain components being configured as part of a distributed computing system.

[0105] The processing system may be configured as a general-purpose processing system with one or more micro-processors providing the processor functionality and external memory providing at least a portion of the machine-readable media, all linked together with other supporting circuitry through an external bus architecture. Alternatively, the processing system may comprise one or more neuro-morphic processors for implementing the neuron models and models of neural systems described. As another alternative, the processing system may be implemented with an application specific integrated circuit (ASIC) with the processor, the bus interface, the user interface, supporting circuitry, and at least a portion of the machine-readable media integrated into a single chip, or with one or more field programmable gate arrays (FPGAs), programmable logic devices (PLDs), controllers, state machines, gated logic, discrete hardware components, or any other suitable circuitry, or any combination of circuits that can perform the various functionality described throughout this disclosure. Those skilled in the art will recognize how best to implement the described functionality for the processing system depending on the particular application and the overall design constraints imposed on the overall system.

[0106] The machine-readable media may comprise a number of software modules. The software modules include instructions that, when executed by the processor, cause the processing system to perform various functions. The software modules may include a transmission module and a receiving module. Each software module may reside in a single storage device or be distributed across multiple storage devices. By way of example, a software module may be loaded into RAM from a hard drive when a triggering event occurs. During execution of the software module, the processor may load some of the instructions into cache to increase access speed. One or more cache lines may then be loaded into a general register file for execution by the processor. When referring to the functionality of a software module below, it will be understood that such functionality is implemented by the processor when executing instructions from that software module. Furthermore, it should be appreciated that aspects of the present disclosure result in improvements to the functioning of the processor, computer, machine, or other system implementing such aspects.

[0107] If implemented in software, the functions may be stored or transmitted over as one or more instructions or code on a computer-readable medium. Computer-readable media include both computer storage media and communication media including any medium that facilitates transfer of a computer program from one place to another. A storage medium may be any available medium that can be accessed by a computer. By way of example, and not limitation, such computer-readable media can comprise RAM, ROM, EEPROM, CD-ROM or other optical disk storage, magnetic disk storage or other magnetic storage devices, or any other medium that can be used to carry or store desired program code in the form of instructions or data structures and that

can be accessed by a computer. Additionally, any connection is properly termed a computer-readable medium. For example, if the software is transmitted from a website, server, or other remote source using a coaxial cable, fiber optic cable, twisted pair, digital subscriber line (DSL), or wireless technologies such as infrared (IR), radio, and microwave, then the coaxial cable, fiber optic cable, twisted pair, DSL, or wireless technologies such as infrared, radio, and microwave are included in the definition of medium. Disk and disc, as used, include compact disc (CD), laser disc, optical disc, digital versatile disc (DVD), floppy disk, and Blu-ray® disc where disks usually reproduce data magnetically, while discs reproduce data optically with lasers. Thus, in some aspects, computer-readable media may comprise non-transitory computer-readable media (e.g., tangible media). In addition, for other aspects computer-readable media may comprise transitory computer-readable media (e.g., a signal). Combinations of the above should also be included within the scope of computer-readable media.

[0108] Thus, certain aspects may comprise a computer program product for performing the operations presented. For example, such a computer program product may comprise a computer-readable medium having instructions stored (and/or encoded) thereon, the instructions being executable by one or more processors to perform the operations described. For certain aspects, the computer program product may include packaging material.

[0109] Further, it should be appreciated that modules and/or other appropriate means for performing the methods and techniques described can be downloaded and/or otherwise obtained by a user terminal and/or base station as applicable. For example, such a device can be coupled to a server to facilitate the transfer of means for performing the methods described. Alternatively, various methods described can be provided via storage means (e.g., RAM, ROM, a physical storage medium such as a compact disc (CD) or floppy disk, etc.), such that a user terminal and/or base station can obtain the various methods upon coupling or providing the storage means to the device. Moreover, any other suitable technique for providing the methods and techniques described to a device can be utilized.

[0110] It is to be understood that the claims are not limited to the precise configuration and components illustrated above. Various modifications, changes, and variations may be made in the arrangement, operation, and details of the methods and apparatus described above without departing from the scope of the claims.

1. An apparatus comprising:
 - at least one memory; and
 - at least one processor coupled to the at least one memory, the at least one processor configured to:
 - receive a machine learning (ML) model, the ML model being represented as a graph having multiple nodes coupled by edges;
 - assign a value number to each node in the graph based on a similarity in characteristics of the multiple nodes;
 - reconstruct a loop for computations in the graph based on the value number to generate a reconstructed loop; and
 - determine loop bounds of the reconstructed loop using an affine scalar evolution analysis technique.

2. The apparatus of claim 1, in which the at least one processor is further configured to group the multiple nodes into a group by applying a hashing function based on a node type.

3. The apparatus of claim 2, in which the group represents identical computation expressions.

4. The apparatus of claim 3, in which the identical computation expressions correspond to iterations of an unrolled loop.

5. The apparatus of claim 1, in which the loop bounds are determined based on a split history offset of each operation in the graph.

6. The apparatus of claim 5, in which the at least one processor is further configured to determine a stride of the reconstructed loop based on a difference between neighboring split history offsets.

7. The apparatus of claim 1, in which the at least one processor is further configured to determine a schedule for performing operations in the ML model using one or more loop optimization techniques on the reconstructed loop.

8. A processor-implemented method performed by one or more processors, the processor-implemented method comprising:

receiving a machine learning (ML) model, the ML model being represented as a graph having multiple nodes coupled by edges;

assigning a value number to each node in the graph based on a similarity in characteristics of the multiple nodes;

reconstructing a loop for computations in the graph based on the value number to generate a reconstructed loop; and

determining loop bounds of the reconstructed loop using an affine scalar evolution analysis technique.

9. The processor-implemented method of claim 8, further comprising grouping the multiple nodes into a group by applying a hashing function based on a node type.

10. The processor-implemented method of claim 9, in which the group represents identical computation expressions.

11. The processor-implemented method of claim 10, in which the identical computation expressions correspond to iterations of an unrolled loop.

12. The processor-implemented method of claim 8, in which the loop bounds are determined based on a split history offset of each operation in the graph.

13. The processor-implemented method of claim 12, further comprising determining a stride of the reconstructed loop based on a difference between neighboring split history offsets.

14. The processor-implemented method of claim 8, further comprising determining a schedule for performing operations in the ML model using one or more loop optimization techniques on the reconstructed loop.

15. An apparatus comprising:

means for receiving a machine learning (ML) model, the ML model being represented as a graph having multiple nodes coupled by edges;

means for assigning a value number to each node in the graph based on a similarity in characteristics of the multiple nodes;

means for reconstructing a loop for computations in the graph based on the value number to generate a reconstructed loop; and

means for determining loop bounds of the reconstructed loop using an affine scalar evolution analysis technique.

16. The apparatus of claim 15, further comprising means for grouping the multiple nodes into a group by applying a hashing function based on a node type.

17. The apparatus of claim 16, in which the group represents identical computation expressions and the identical computation expressions correspond to iterations of an unrolled loop.

18. The apparatus of claim 15, in which the loop bounds are determined based on a split history offset of each operation in the graph.

19. The apparatus of claim 18, further comprising means for determining a stride of the reconstructed loop based on a difference between neighboring split history offsets.

20. The apparatus of claim 15, further comprising means for determining a schedule for performing operations in the ML model using one or more loop optimization techniques on the reconstructed loop.

* * * * *